

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA0607

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model, (BL 6)

Assessment Tool Weightage: 40%

QUESTIONS: 02

Total Marks:20

Annexure A

Experiments

Code of Conduct:

I VIMALA K 192424276 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student :

Vimala K

Total Marks

Criteria	Conceptual Understanding	Accuracy of Answers	Application of Knowledge	Completeness of Response	Clarity & Presentation	Total
Weightage	30%	25%	15%	15%	15%	
Q1						
Q2						

Annexure A

Short Answer Test

12. Perform an experiment using a **Principal Component Analysis (PCA)** technique for dimensionality reduction. Apply PCA on a high-dimensional dataset and vary the number of principal components retained. Evaluate the impact on model performance using accuracy and computational efficiency. Analyze how variance retention influences information loss and classification results.

Principal Component Analysis (PCA) for Dimensionality Reduction

Aim

To perform **Principal Component Analysis (PCA)** for dimensionality reduction on a high-dimensional dataset by varying the number of principal components and evaluating its impact on **classification accuracy, variance retention, and computational efficiency**.

Objective

- To understand the concept of PCA in Machine Learning.
- To reduce the dimensionality of a high-dimensional dataset.
- To vary the number of principal components retained.
- To measure the percentage of variance retained.
- To compare classification accuracy before and after PCA.
- To analyze computational efficiency and information loss.

Theory

Principal Component Analysis (PCA) is an unsupervised dimensionality-reduction technique used to transform a large number of correlated features into a smaller set of **uncorrelated principal components**.

Each principal component represents a direction of maximum variance in the dataset.

The first principal component captures the maximum variance, the second captures the next highest variance, and so on.

The amount of information retained can be calculated using:

Variance Retention = Sum of selected eigenvalues / Sum of all eigenvalues × 100

If more principal components are retained:

- More information is preserved.
- Classification accuracy generally increases.
- Computational cost may increase.

If fewer components are retained:

- Dimensionality and computation decrease.
- Some information is lost.
- Classification accuracy may decrease.

Dataset

For this experiment, the **Breast Cancer Wisconsin dataset** available in sklearn can be used. It contains:

- **569 samples**
- **30 input features**
- **2 target classes**

Since it has 30 features, PCA can be applied to reduce the number of dimensions.

Algorithm

1. Load the high-dimensional dataset.
2. Separate the features and target variable.
3. Split the dataset into training and testing data.
4. Standardize the features.
5. Apply PCA with different numbers of components.
6. Transform the training and testing data.
7. Train a classification model such as Logistic Regression.
8. Calculate classification accuracy.
9. Measure computational time.
10. Compare variance retention, accuracy, and execution time.
11. Analyze the effect of dimensionality reduction.

Python Program

```
import timeimport numpy as npimport pandas as pdimport matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancerfrom sklearn.model_selection import
train_test_splitfrom sklearn.preprocessing import StandardScalerfrom sklearn.decomposition
import PCAfrom sklearn.linear_model import LogisticRegressionfrom sklearn.metrics import
accuracy_score
X = data.data
y = data.target
print("Original number of features:", X.shape[1])print("Number of samples:", X.shape[0])
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2,
                                                    random_state=42,
                                                    stratify=y)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)X_test_scaled = scaler.transform(X_test)
model = LogisticRegression(max_iter=5000)model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)
baseline_accuracy = accuracy_score(y_test, y_pred)baseline_time = time.time() - start
print("\nWithout PCA")print("Accuracy:", baseline_accuracy)print("Time:", baseline_time)
```

```

results = []
for n in components_list:

    start = time.time()

    pca = PCA(n_components=n)

    X_train_pca = pca.fit_transform(X_train_scaled)
    X_test_pca = pca.transform(X_test_scaled)

    model = LogisticRegression(max_iter=5000)
    model.fit(X_train_pca, y_train)

    y_pred = model.predict(X_test_pca)

    accuracy = accuracy_score(y_test, y_pred)

    execution_time = time.time() - start

    variance_retained = np.sum(pca.explained_variance_ratio_) * 100

    results.append([
        n,
        variance_retained,
        accuracy * 100,
        execution_time
    ])
    results,
    columns=[
        "Components",
        "Variance Retained (%)",
        "Accuracy (%)",
        "Time (seconds)"
    ]
)
print("\nPCA Results")print(results_df.to_string(index=False))
    results_df["Components"],
    results_df["Variance Retained (%)"],
    marker="o"
)plt.xlabel("Number of Principal Components")plt.ylabel("Variance Retained (%)")plt.title("PCA - Variance Retention")plt.grid()plt.show()
    results_df["Components"],
    results_df["Accuracy (%)"],
    marker="o"
)plt.xlabel("Number of Principal Components")plt.ylabel("Accuracy (%)")plt.title("PCA - Classification Accuracy")plt.grid()plt.show()
    results_df["Components"],
    results_df["Time (seconds)"],
    marker="o"

```

```
)plt.xlabel("Number of Principal Components")plt.ylabel("Time (seconds)")plt.title("PCA - Computational Efficiency")plt.grid()plt.show()
```

Expected Output

The exact values may differ slightly depending on the train-test split and system, but the output will have a structure like this:

Principal Components	Variance Retained	Accuracy	Computational Time
2	Lower	Lower	Low
5	High	Improved	Low
10	Very High	High	Low
15	Very High	High	Low
20	Very High	High	Moderate
25	Almost all	Very High	Moderate
30	100%	Similar to original	Higher

PCA: Variance Retention

Analysis

1. Effect of number of components

When only a small number of principal components such as **2 or 5** are retained, the dimensionality is significantly reduced. However, some useful information may be lost, resulting in lower classification accuracy.

As the number of components increases, more information from the original dataset is retained. Therefore, classification accuracy generally improves.

2. Effect of variance retention

Variance retention represents how much information from the original features is preserved.

For example:

- Low variance retention → greater information loss.
- High variance retention → less information loss.
- 100% variance retention → no information loss due to PCA transformation.

Therefore, selecting an appropriate number of components is important.

3. Effect on accuracy

PCA may reduce accuracy when too few components are selected because important information is discarded.

However, after retaining sufficient components, the accuracy can become close to the original model.

4. Effect on computational efficiency

The original dataset contains **30 features**. PCA can reduce this to 5, 10, or 15 features.

Fewer features mean:

- Less data processing.
- Lower memory requirements.
- Faster model training.
- Faster prediction in many cases.

Thus, PCA provides a trade-off between **information retention and computational efficiency**.

Result

PCA successfully reduced the dimensionality of the high-dimensional dataset. Different numbers of principal components were tested and their effects on **variance retention, classification accuracy, and computational efficiency** were analyzed.

It was observed that retaining more principal components preserves more information and generally provides better classification performance, while retaining fewer components provides greater dimensionality reduction and computational efficiency.

Conclusion

Principal Component Analysis is an effective dimensionality-reduction technique in Machine Learning. The experiment demonstrates that the number of principal components directly affects information retention and classification performance. A suitable number of components should be selected to achieve a good balance between **accuracy, information preservation, and computational efficiency**.

13. Implement a **Gradient Descent optimization algorithm** and conduct experiments by varying learning rates and initialization values. Observe convergence behavior and record the number of iterations required to reach optimal values. Analyze cases of slow convergence or divergence, and suggest improvements such as momentum or adaptive learning rate methods.

Gradient Descent Optimization Algorithm

Aim

To implement the **Gradient Descent optimization algorithm** and study the effect of different **learning rates and initialization values** on convergence. The number of iterations required to reach the optimal value is recorded, and cases of slow convergence and divergence are analyzed.

Objective

- To understand the working principle of Gradient Descent.
- To implement Gradient Descent without using an optimization library.
- To experiment with different learning rates.
- To experiment with different initial values.
- To observe convergence and divergence behavior.
- To calculate the number of iterations required to reach the optimum.
- To understand improvements using **Momentum** and **Adaptive Learning Rate** methods.

Theory

Gradient Descent is an iterative optimization algorithm used in Machine Learning to minimize a loss or cost function.

The parameter is updated using:

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha d\theta dJ$$

Where:

- θ = parameter/weight
- α = learning rate
- $J(\theta)$ = cost function
- $d\theta dJ$ = gradient

For this experiment, consider the simple convex function:

$$J(x) = (x-3)^2$$

The minimum value occurs at:

$$x=3$$

The derivative is:

$$dx dJ = 2(x-3)$$

Therefore, the Gradient Descent update becomes:

$$x_{\text{new}} = x - \alpha[2(x-3)]$$

Algorithm

1. Define the cost function $J(x)=(x-3)^2$.
2. Calculate its gradient.
3. Select an initial value of x .
4. Select a learning rate.
5. Calculate the gradient.
6. Update x .
7. Calculate the new cost.
8. Repeat until the change becomes smaller than the tolerance.
9. Record the number of iterations.
10. Repeat the experiment for different learning rates and initialization values.
11. Compare convergence, slow convergence, and divergence.
12. Suggest Momentum and Adaptive Learning Rate as improvements.

Python Program

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def cost_function(x):
    return (x - 3) ** 2
def gradient(x):
    return 2 * (x - 3)

x = x_initial
history = []

for iteration in range(1, max_iterations + 1):

    cost = cost_function(x)
    history.append(cost)

    grad = gradient(x)

    x_new = x - learning_rate * grad
    if abs(x_new - x) < tolerance:
        x = x_new
        history.append(cost_function(x))
        return x, cost_function(x), iteration, history

    x = x_new

if abs(x) > 1e10:
    return x, np.inf, iteration, history

return x, cost_function(x), max_iterations, history
```

```

results = []
for lr in learning_rates:

    for initial in initial_values:

        final_x, final_cost, iterations, history = gradient_descent(
            initial, lr
        )

        if np.isfinite(final_cost) and final_cost < 1e-6:
            status = "Converged"
        elif np.isfinite(final_cost):
            status = "Slow / Not Converged"
        else:
            status = "Diverged"

        results.append([
            lr,
            initial,
            final_x,
            final_cost,
            iterations,
            status
        ])
results_df = pd.DataFrame(
    results,
    columns=[
        "Learning Rate",
        "Initial Value",
        "Final x",
        "Final Cost",
        "Iterations",
        "Status"
    ]
)
print("Gradient Descent Experimental Results")print(results_df.to_string(index=False))

```

Graph 1: Convergence for Different Learning Rates

Use the following code to visualize how the cost function decreases for different learning rates.

```

plt.figure(figsize=(8, 5))
initial_value = 10
for lr in [0.01, 0.1, 0.5, 0.9]:
    _, _, _, history = gradient_descent(
        initial_value,
        lr,

```

```

        max_iterations=100
    )

    plt.plot(
        history,
        label=f"Learning Rate = {lr}"
    )
plt.xlabel("Iteration")plt.ylabel("Cost J(x)")plt.title("Gradient Descent
Convergence")plt.legend()plt.grid()plt.show()

```

Graph Representation
Gradient Descent Convergence

Illustrative decrease in cost for different learning rates starting from the same initial value.

Learning rate 0.01Learning rate 0.05Learning rate 0.1Learning rate 0.5
01530456001020304050

Interpretation:

- **Learning rate = 0.01:** Very slow convergence.
- **Learning rate = 0.05:** Converges gradually.
- **Learning rate = 0.1:** Faster convergence.
- **Learning rate = 0.5:** Reaches the minimum very quickly for this function.

Graph 2: Learning Rate vs Number of Iterations

```

convergence_results = []
for lr in learning_rates:

    x, cost, iterations, history = gradient_descent(
        10,
        lr
    )

    convergence_results.append([
        lr,
        iterations,
        x,
        cost
    ])
convergence_df = pd.DataFrame(
    convergence_results,
    columns=[
        "Learning Rate",
        "Iterations",
        "Final x",
        "Final Cost"
    ]
)

```

```

    ]
)
print(convergence_df)
plt.figure(figsize=(8, 5))
plt.plot(
    convergence_df["Learning Rate"],
    convergence_df["Iterations"],
    marker="o"
)
plt.xlabel("Learning Rate")plt.ylabel("Number of Iterations")plt.title("Learning Rate vs
Number of Iterations")plt.grid()plt.show()

```

Graph Representation

Learning Rate vs Iterations

Illustrative relationship between learning rate and iterations needed for convergence.

02004006008000.010.050.10.50.9

Experimental Results

A typical result table can be represented as:

Learning Rate	Initial Value	Behavior	Convergence
0.01	10	Very slow	Yes
0.05	10	Slow	Yes
0.10	10	Fast	Yes
0.50	10	Very fast	Yes
0.90	10	Oscillatory/fast	Yes
1.10	10	Divergence	No

Effect of Initialization Values

The initial value determines where Gradient Descent starts.

For a simple convex function such as:

$$J(x)=(x-3)^2$$

different initial values such as **-10, 0, 5, and 10** will eventually converge to $x=3$ when an appropriate learning rate is used.

Therefore, for this simple function, initialization mainly affects **how far the algorithm must travel**, rather than the final optimum.

Analysis

1. Small Learning Rate

When the learning rate is very small, the updates are also small.

Result:

The algorithm converges, but requires many iterations.

Example:

Learning rate = **0.01**

Problem: Slow convergence.

2. Suitable Learning Rate

A suitable learning rate allows the algorithm to move quickly toward the minimum without overshooting.

Example:

Learning rate = **0.1 or 0.5**

Result: Fast convergence and fewer iterations.

3. Large Learning Rate

When the learning rate becomes too large, the algorithm can overshoot the minimum.

This can result in:

- Oscillation
- Instability
- Increasing cost
- Divergence

For this function, learning rates greater than approximately **1.0** lead to divergence.

4. Effect of Initialization

Different starting points produce different optimization paths.

However, because the selected function is convex, the algorithm reaches the same global minimum when the learning rate is appropriate.

Improvements

A. Momentum

Momentum uses information from previous updates to accelerate Gradient Descent.

The update can be represented as:

$$v_t = \beta v_{t-1} + \alpha \nabla J(\theta) \quad \theta = \theta - v_t$$

Advantages:

- Faster convergence.
- Reduces oscillation.
- Helps move through shallow regions.
- Useful for complex optimization problems.

B. Adaptive Learning Rate

Adaptive methods automatically adjust the learning rate during training.

Examples include:

- **AdaGrad**
- **RMSProp**
- **Adam**

These methods can provide faster and more stable convergence without manually selecting a single fixed learning rate.

Result

The Gradient Descent algorithm was successfully implemented and tested using different **learning rates and initialization values**.

The experiment showed that:

- Very small learning rates cause **slow convergence**.
- Appropriate learning rates produce **fast convergence**.
- Very large learning rates can cause **oscillation or divergence**.
- Different initialization values affect the optimization path and number of iterations.
- Momentum and adaptive learning-rate methods can improve convergence speed and stability.

Conclusion

Gradient Descent is an important optimization technique used in Machine Learning for minimizing loss functions. The **learning rate is a critical parameter** because it controls the size of each update. Selecting an appropriate learning rate provides a balance between **convergence speed and stability**. Techniques such as **Momentum, Adam, RMSProp, and AdaGrad** can further improve the optimization process.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand